Can LLMs Build Better Annotators? Self-Improving Harnesses for Ontology-Guided Annotation

Anonymous ACL submission

Abstract

Rich annotation schemes specify structures and conventions that a general-purpose LLM may not follow. We study whether supervised revision of a modular annotation harness can improve annotation from a few gold documents without changing model weights. AnnoForge records step traces, localizes discrepancies using structured gold alignment, and applies typed, anchored edits to the instructions of designated skills. Proposed tools and steps undergo machine checks and a separate model review. In one fixed-order Chinese UMR run, a paired test-then-train evaluation gives a mean gain of 9.0 sentence-graph F-score points over the seed on 9 previously untrained-on documents (document bootstrap 95% interval 5.5–12.5). Gains concentrate in convention-sensitive checks; little change is observed in sense, aspect and core-argument checks. Stage-wise scores associate most of the gain with the generator, while the checking stage adds 0.5 points. No tool proposal is admitted. Instructions grow 39-fold without a corresponding increase in sequential gains, and a separate 9B joint builder–annotator configuration degrades. These are observations from one trajectory, rather than evidence of causal attribution or broad superiority. Matched held-out baselines, design ablations and repeated runs remain pending.

1 Introduction

Structured annotation follows a scheme of labels, relations and conventions: UMR graphs (Van Gysel et al., 2021; Bonn et al., 2024), TimeML (Pustejovsky et al., 2003), or political codebooks (Lehmann et al., 2026). Even revising model drafts requires substantial human work (Sun et al., 2024). General-purpose LLMs may recognize semantic relations yet fail to follow the scheme. An annotation *harness* supplies step instructions, tools, call chains and deterministic repairs; its construction typically requires repeated error analysis.

We ask whether an LLM can improve a manually initialized harness from a few gold documents. Guideline refinement revises instruction text against gold (Kim et al., 2026); agent optimization can revise prompts and workflows (Zhang et al., 2026a; Hu et al., 2025). Structured annotation supplies aligned local discrepancies in addition to a scalar score, but identifying a useful repair target, avoiding instance-specific edits and evaluating transfer remain difficult.

AnnoForge (Figure 1) records step traces, assigns candidate edit targets, and applies typed, anchored instruction edits. Proposed tools and steps require machine checks and a separate model review. We evaluate test-then-train: a document is scored before its use as builder training data, paired with the seed on that same document. This operational localization is not verified causal attribution.

In one fixed-order Chinese UMR run (11 documents, 415 sentences):

- The learned harness gains 9.0 sentence-graph F-score points over the seed on 9 paired documents (document bootstrap interval 5.5–12.5). Matched held-out baselines remain pending.
- Gains concentrate in convention-sensitive checks. Most are present at the generator stage; the checker adds 0.5 points. This does not isolate each component's causal contribution.
- Instructions grow from 11.2 to 434 KB; 1 of 1060 applied edits delete text. No tool proposal is admitted. A separate 9B joint builder–annotator configuration degrades.

Our contribution is an editing implementation and a paired sequential diagnosis of transfer, instruction growth and failed extension proposals. Self-improvement here is supervised artifact revision, not learning without gold feedback. Additional task results and component ablations are pending.

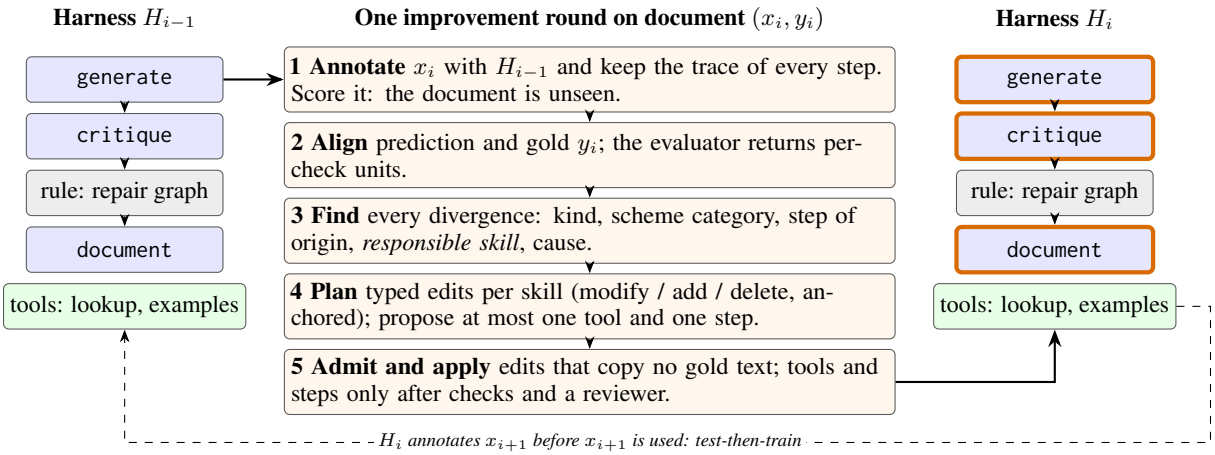


Figure 1: AnnoForge. A harness is a chain of skills (instruction text with optional examples), shared tools and deterministic rule steps. One round annotates a gold document, attributes each divergence from gold to the responsible skill and scheme category, and edits those skills (orange borders). The next document is annotated by the new harness before it is used to improve it.

Scope and shared infrastructure. The companion procedural study (Anonymous, 2026) examines deterministic dispatch and assembly around fragment or closed decisions. We instead study instruction revision in a three-call whole-graph harness. The studies share UMR data, alignment and judge infrastructure, and our procedural comparisons reuse its runs; shared runs are not independent replication.

2 Related Work

LLMs as annotators. LLMs match or replace crowd workers on many labeling tasks (Gilardi et al., 2023; He et al., 2024; Ziems et al., 2024; Tan et al., 2024) but follow a scheme's conventions poorly out of the box: as AMR analyzers (Ettinger et al., 2023), with concept guidelines (Fonseca and Cohen, 2024), and with real codebooks (Halterman and Keith, 2026). Automatic UMR parsers build on AMR parsers (Chun and Xue, 2024; Markle et al., 2026).

Guidelines as input and as learnable objects. Guidelines help when models are trained to follow them (Sainz et al., 2024; Srivastava et al., 2025) or replace examples in context (Huang et al., 2025); Guideline Learning synthesizes rules from errors for in-context extraction (Pang et al., 2023), a modern form of error-driven rule learning (Brill, 1995). Earlier work lets LLMs improve annotation guidelines against gold (Bibal et al., 2025; Kim et al., 2025, 2026): for biomedical NER, a moderator picks the most frequent error pattern, writes one principle, re-emits the whole guideline, and keeps it if development F1 improves (Kim et al., 2026). Codebook work revises coding instructions from model disagreement or expert feedback (He et al., 2026; Yuan et al., 2026). The cited guideline-refinement methods primarily change instruction text; we change a harness and assign candidate edit targets to components before editing them.

Prompt and context optimization. Instructions can be searched (Zhou et al., 2023; Yang et al., 2024), edited along textual "gradients" (Pryzant et al., 2023; Yuksekgonul et al., 2025), compiled with demonstrations (Khattab et al., 2024; Opsahl-Ong et al., 2024), evolved by reflection (Agrawal et al., 2026), or grown as playbooks and memories (Zhang et al., 2026c; Suzgun et al., 2026). Their feedback ranges from scalar task scores to execution traces and textual critiques, and with little data can lose to fixed prompts (Singh, 2026); our setting has one document per step but a full gold structure per document.

Self-improving agents and harnesses. Self-feedback rarely helps without an external signal (Madaan et al., 2023; Shinn et al., 2023; Huang et al., 2024; Kamoi et al., 2024). Agents learn skills (Wang et al., 2024; Zheng et al., 2025), make tools (Cai et al., 2024; Qian et al., 2023; Wölflein et al., 2025), and search over workflows and their own code (Hu et al., 2025; Zhang et al., 2025, 2026b). Recent work optimizes the harness itself (Lee et al., 2026; Zhang et al., 2026a; Lin et al., 2026a; Xia et al., 2026), admitting edits through regression tests, verified predictions or critics; gains com-

pound only with regression control (Wang et al., 2026), and writing useful updates and benefiting from them dissociate across model tiers (Lin et al., 2026b). These target coding, terminal and web agents with pass/fail rewards; we focus on structured annotation, where aligned gold provides local diagnostic units rather than only a pass/fail reward. Our test-then-train protocol follows stream learning (Dawid, 1984; Gama et al., 2013).

3 Building an Annotator

Schemes, evaluators, harnesses. A scheme fixes the labels, structures and conventions of an annotation; for UMR, concepts with PropBank senses (Palmer et al., 2005), roles, attributes such as aspect and modal strength, named-entity structures, and document-level temporal, modal and coreference relations (Van Gysel et al., 2021). An evaluator compares an annotation with gold and returns, besides a score, *units*: per-check verdicts with the aligned gold and predicted elements. For UMR we use AnCast++ (Sun and Xue, 2024; Sun et al., 2025) for score and alignment and a judge that sorts aligned elements into 25 checks in 11 groups named after parts of the scheme (Appendix A). An annotator is an LLM inside a harness $H = (S, T, G, R)$: *skills* S (an instruction, optional examples and an output schema, run as one model call), *tools* T any skill may call, a *chain* G saying which skill runs on whose output, and *rule steps* R, deterministic code. Running H on a document x yields an annotation $A_H(x)$ and a trace of every step.

The building task. A builder receives a seed harness H_0, manually initialized from the scheme (what the output contains, its format, the checks the evaluator makes, no examples), and gold documents $(x_1, y_1), \ldots, (x_n, y_n)$, and returns H_n. The only signal is gold; no person edits the harness (one disclosed exception, §4.1). A_{H_n} should score well on unseen documents at a known cost, and H_n should stay readable for annotators to audit.

Sequential evaluation. With a few dozen gold documents there is no room for a large development set, and a document used for improving cannot also measure. We evaluate test-then-train, as for stream learners (Dawid, 1984; Gama et al., 2013): for $i = 1, \ldots, n$, annotate x_i with H_{i-1} and score it, then improve H_{i-1} into H_i with (x_i, y_i). Every score precedes using that document as builder training data, and documents differ in difficulty, so each score is paired with H_0 on the same document. A planned held-out set not used for building this harness will compare the final harness with baselines that receive the same gold.

4 AnnoForge

4.1 Harness representation

Every part of a harness is a file that a builder can read, edit and diff. A skill is a Markdown instruction whose last section holds the per-call input, so the part above it is identical across calls and cached; it may have worked examples in a separate file read through the shared `get_example` tool. Tools are typed Python functions with a docstring. A registry declares skills, output schemas and the chain; rule steps are Python functions that call no model. Versions are directories never edited after their round: round y on document x produces R_{xy}, and $H_n = R_{n3}$ is the version after document n.

The UMR harness. Per sentence, three model calls (Figure 1). `generate` writes the whole sentence graph from a 2 KB description of what a UMR graph contains and how it is written, with no examples. `critique` checks the draft one evaluator group at a time (nodes, senses, attributes, arguments, role labels, names, reentrancy); each group says what the evaluator checks and lists *common errors*, a few generic ones in the seed. It returns findings and a corrected graph. A rule step repairs the last parseable graph (duplicate or undefined variables, self-loops, disconnected parts).[1] `document` sees the graph and the five before it and writes the sentence's temporal, modal and coreference relations. All skills can call `propbank_lookup` and `get_example`. The seed has 11.2 KB of instructions.

4.2 One improvement round

1. Annotate and trace. The current harness annotates the gold document; the trace holds each sentence's draft, each later step's findings and the triples it added or removed, the rule repairs, and the document-level calls.

2. Align. The evaluator aligns prediction and gold and lists the units it did not count as correct.

[1] When R_{12} annotated document 0001, one sentence lost all credit to a missing bracket, and we added bracket balancing to this rule from R_{13} on: the only change a person made to any harness during the run.

3. Find. For every sentence with an error, the builder model sees the sentence, both graphs, the alignment, the incorrect units, both sides' document-level relations, and the trace. It lists every divergence, merging those with one cause, and labels each with its kind, its evaluator check, the step it came from, the skill whose instructions *should change*, its cause, and, if gold follows a convention, that convention in general terms. The prompt gives the rule for blame: what `critique` should catch in its group is `critique`'s; a draft no check can repair, or a misleading instruction of `generate`, is `generate`'s; document-level errors are `document`'s.

4. Plan. One call reads up to 250 problems (taken in turn from each sentence), grouped by responsible skill and, for `critique`, by evaluator group, together with every skill's full text. It returns edits, each naming a skill, a file, an action, an anchor, a text and the problems it addresses: *modify* replaces an instruction, common error or example that is wrong or too narrow; *add* inserts a missing rule, example or common error after an anchor; *delete* removes text that causes an error or contradicts gold. Edits must be general and checkable, one edit should fix many problems, and one-offs and gold errors get no edit; changes outside the instructions go to a list of suggestions the loop does not apply.

5. Admit and apply. An edit applies only if its anchor occurs exactly once, and is rejected if it shares eight consecutive characters with any sentence of the corpus, training or held-out. The automatic guard therefore accesses held-out source text as a rejection filter, although the builder does not receive held-out gold annotations or scores; it is not a completely sealed test interface. A round may also propose one tool (a Python function every skill can call) and one step (a skill inserted in the chain). A proposal must pass machine checks (the registry loads; a tool imports only whitelisted modules, touches no files, has one typed public function, and passes at least three tests of its own in an isolated process) and then a reviewer, a separate builder call that approves only if six criteria clearly hold: the problem recurs and is costly, no edit could fix it as well, the part has one narrow responsibility, it is worth its cost, it is correct, and it leaks nothing (Appendix C). Edits that depend on a rejected proposal are dropped.

Schedule. Documents are processed in order, three rounds each; each round re-annotates the document with the previous round's version. H_n then annotates document $n{+}1$, unchanged; that is its test score.

Trace-based edit-target assignment (step 3) and typed, anchored edits (step 4) support inspection and auditability: each edit touches one place and leaves a diff, instead of a rewritten guideline. The selected edit target is not a verified causal source: upstream decisions and alignment errors can affect downstream units. Attribution accuracy and its separate contribution require an ablation. Tools and steps are gated because they persist across documents. In this version the loop does not test an edit before keeping it, and nothing prompts it to consolidate; §6 shows the consequences. TimeML and manifesto coding use related harness variants with different admission and edit policies (Appendix F).

5 Experimental Setup

Data. Chinese UMR (Bonn et al., 2024), 31 documents with sentence graphs and document-level relations, in a cleaned version of the gold [cite the cleanup]. The sequential run takes documents 0001–0011 in numerical order (415 sentences, 10–82 per document) and stopped after the first round on 0011, so 9 documents (0002–0010) have both a learned and a seed annotation. The other 20 documents (297 sentences) were not used to train the main UMR harness and form its planned held-out set. Some documents were used by other model or procedural-harness experiments; this is a run-specific split, not a project-wide untouched set. Cross-system comparisons must audit those exposures before using a common test set.

Models and metrics. Claude Sonnet 5.5 annotates and Claude Opus 5.5 builds (find, plan, review), with adaptive thinking as recorded in the run notes; exact historical backend and effort settings remain to be verified [G14]; §6.6 varies them. AnCast++ (Sun et al., 2025) aligns each predicted sentence graph with gold and gives F over sentence graphs (F) and a comprehensive score that adds document-level modal, temporal and coreference relations (C). For analysis we use the judge's per-check scores and per-sentence Smatch.

Baselines. Each sees the same gold documents as the builder, or none. **Zero-shot prompt**: one call per sentence naming the language and the scheme

and pointing to its guidelines; sentence graphs only. **Seed harness** H_0. **Few-shot** [to run]: H_0 with the gold graphs of the k most similar training sentences (character TF-IDF) in `generate`, $k \in \{10, 30\}$. **Guideline-only refinement** [to run]: the closest analogue of earlier work (Kim et al., 2026), with the same builder and gold, but a single guideline prompt rewritten in full each round from the dominant error pattern and kept only if the training document improves. **Procedural harness**: a hand-designed harness that follows the human annotation procedure (Anonymous, 2026), with dozens of narrow decisions per sentence, improved by the same builder with edits confined to the skill that made each decision.

Statistics. We report the mean paired difference over documents with a bootstrap 95% interval (20,000 resamples), the exact Wilcoxon signed-rank test, and the number of documents that improve; sentence-level intervals resample documents, then sentences. These document-level summaries are conditional on one fixed-order trajectory: successive harnesses share training history, and the intervals and tests do not estimate variation across builder runs or document orders. The sentence-level bootstrap concerns Smatch and includes only sentences with non-missing scores from both systems; changes above $+0.01$ or below -0.01 count as improvements or declines. Missing-score counts and their causes require an offline denominator audit [G14]. Each configuration was run once [G-var].

6 Results

6.1 The learned harness is better on unseen documents

On the documents that also have the zero-shot prompt (0002–0007; Table 1), the systems form a ladder: 30.9 F for one call per sentence, 46.9 for the seed harness, 58.5 for the learned harness. Over the 9 documents with a seed control, the learned harness gains 9.0 F (95% CI 5.5–12.5; Wilcoxon $p = 0.008$) and improves 8 of them; on the comprehensive score it gains 10.1 (7.0–13.0) and improves all 9 ($p = 0.004$). Of 337 complete-case paired sentences, 77.2% improve and 18.4% decline by more than one Smatch point (mean +9.5, 6.2–12.5). The one document that does not improve, 0008 (-0.8), has ten sentences.

Training fit is not generalization. Three rounds on a document raise its own F by 26.6 points, to 84.2 on average and up to 99.2, against 9.0 on the next, unseen document (Figure 2a). The leakage guard restricts some verbatim overlap, not learning one document's particular conventions. Evaluated on its training documents, the same loop would look three times better.

6.2 Against baselines with the same gold

The held-out comparison is pending (Table 2). One matched comparison exists: trained on 0001–0004 with the same models, the learned harness scores 58.5 F on document 0005 and the procedural harness 55.7; the procedural seed starts higher there (51.1 vs. 48.0), so the learned gains are +10.5 and +4.6 (one document; the procedural harness had 10 rounds, ours 12; Appendix Table 11).

6.3 Where the gains come from

Stage-wise generator and checker scores. Scoring the graph after each step (Appendix Table 9) shows that the learned `generate` alone is 9.6 F better than the seed's on unseen documents, and `critique` adds 0.5 on top (-0.4 to +1.4), less than the 1.0 it adds to the seed's draft; on the document it was trained on it adds 1.7. Most of the observed sequential gain is already present at the generator stage. These stage-wise comparisons do not isolate the causal contribution of updating each skill, because the generator and checker were revised jointly. A variant with five specialized checks shows this more sharply: the checks add +8.9 to +9.3 on their training documents and 0.5 on unseen ones (Appendix Table 10). At annotation time a checker has no gold, only what it was told, as self-correction studies would predict (Huang et al., 2024; Kamoi et al., 2024).

Gains by evaluator check. By judge check (Figure 3), the largest gains are on the scheme's conventions: pronoun person and number (+24.8 points), the author's modal strength (+18.9), non-core role labels (+18.3), arguments of abstract rolesets such as `have-org-role-91` (+13.2), non-core arguments of events (+12.4), entity concepts (+10.7). Little change is observed on several checks involving semantic judgments: PropBank sense (+0.2), aspect (+0.4), core arguments of events (+0.3). Coreference (+5.1) gains little and polarity falls (-10.9, few units). This pattern is consistent with learning scheme conventions, but these checks also

Doc	Sent.	Annotated by	Zero-shot prompt F	Seed H_0 F	Learned F	Δ F	Seed C	Learned C	Δ C	Trained on it, F
0001	51	H_0	31.0	45.1	–	–	40.5	–	–	77.5
0002	60	H_1	31.1	49.4	61.1	+11.8	44.1	56.2	+12.1	77.3
0003	31	H_2	32.3	45.3	57.2	+11.9	41.3	55.4	+14.1	84.4
0004	59	H_3	33.5	42.9	52.4	+9.5	39.3	49.7	+10.4	72.4
0005	40	H_4	32.3	48.0	58.5	+10.5	44.4	57.1	+12.7	79.7
0006	35	H_5	23.6	49.3	56.1	+6.8	40.4	49.2	+8.8	81.0
0007	82	H_6	32.6	46.4	65.5	+19.1	42.3	59.6	+17.3	85.4
0008	10	H_7	TBD	57.1	56.4	−0.8	52.5	53.7	+1.2	92.7
0009	10	H_8	TBD	59.7	68.0	+8.3	55.2	65.3	+10.1	99.2
0010	12	H_9	TBD	51.2	55.2	+4.0	45.7	50.2	+4.5	92.5
0011	25	H_{10}	TBD	TBD	53.4	TBD	TBD	48.5	TBD	–
Mean, 9 paired documents			–	49.9	58.9	+9.0	45.0	55.2	+10.1	
Mean, 0002–0007			30.9	46.9	58.5	+11.6				

Table 1: Sequential (test-then-train) evaluation on Chinese UMR. Document n is annotated by the harness built from documents $1..n-1$ ("Learned"), then used to improve it. "Seed" is H_0 on the same document, the paired control. "Zero-shot prompt" is one call per sentence with no harness (sentence graph only, so only F is comparable). F = AnCast++ sentence F; C = comprehensive score (sentence graphs plus document-level relations). The last column is the harness trained on the document itself (training fit, not generalization). Document 0001 has no earlier harness; 0011 has no seed run yet; zero-shot is missing for 0008–0011.

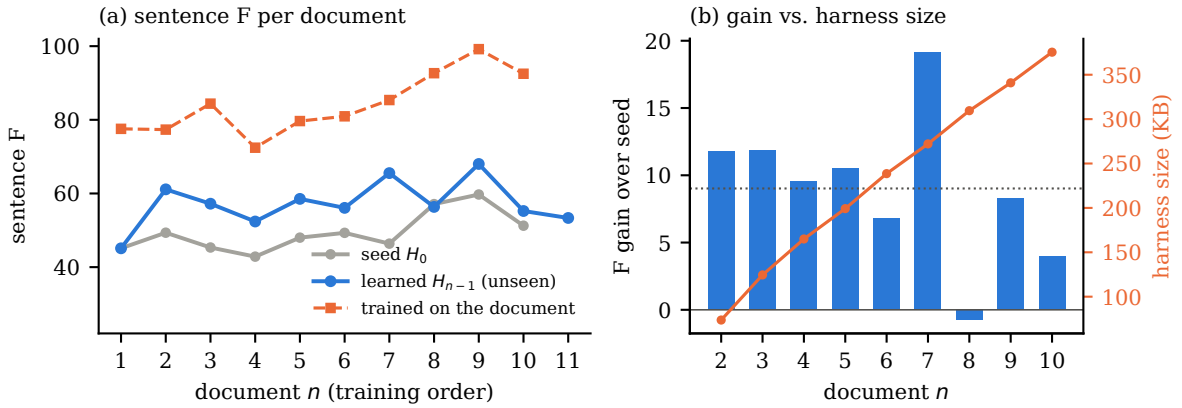


Figure 2: (a) Sentence F per document in training order: seed harness, learned harness that has not seen the document, and the harness after three rounds on it. (b) Gain of the learned harness over the seed per unseen document (bars; dotted: mean) and the size of the harness that annotated it (line).

System (gold: 0001–0004)	F	C	Calls/sent.
Zero-shot prompt	TBD	–	1
Few-shot, k=30	TBD	TBD	3
Guideline-only refinement	TBD	TBD	1
Procedural harness	TBD	TBD	dozens
Seed harness H_0	TBD	TBD	3
Learned H_4 (and H_1, H_{10})	TBD	TBD	3

Table 2: Planned held-out set (20 documents not used to train this harness), same gold budget for all trained systems. [G1–G4]

involve semantic judgments. It does not establish an unchanged semantic capability or a capability ceiling; category counts and repeated, targeted comparisons are needed to test that interpretation.

6.4 Tools are proposed rarely and admitted never

In 31 rounds the builder proposed 3 tools and no steps; the reviewer rejected all three. `check_umr_graph` checked attribute values, entity types and role names; the reviewer kept the purely mechanical checks (colon-prefixed or misspelled closed-list values, parse failures, duplicate variables) and judged the rest to be missing conventions that belong in the instructions. `check_umr_form` flagged bare constants and aspect on discourse role-sets; the second part contradicted the builder's own earlier rule. `check_doc_relations` checked coreference on name nodes and double temporal parents, but one test contradicted a convention of `document`.

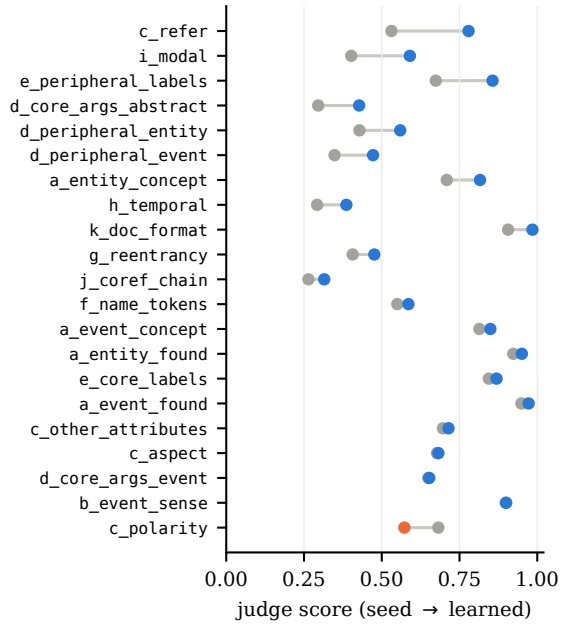


Figure 3: Judge score per check on unseen documents: seed (grey) and learned harness (blue; orange where lower), mean over 9 documents.

Builder / annotator	Harness	n	Seed	Learned	Δ
Opus / Sonnet 5.5	ours^s	9	49.9	58.9	+9.0
Opus / Sonnet 5.5	proc.	2	45.5	52.9	+7.4
DeepSeek (no thinking)	proc.	5	32.5	37.6	+5.2
Qwen3.5-9B	proc.^{s}	13	17.6	16.3	−1.4
Qwen3.5-27B	proc.^{s}	TBD	TBD	TBD	TBD
Opus / small model	ours	TBD	TBD	TBD	TBD
Small model / Sonnet	ours^s	TBD	TBD	TBD	TBD

Table 3: Seed vs. learned harness on unseen documents, sentence F. One name: the same model builds and annotates. proc.: procedural harness. ^{s}sequential protocol; otherwise a few training documents and separate test documents. DeepSeek's seed reaches 49.1 F with thinking on vs. 35.9 off (document 0002).

Variant	ΔF seq.	ΔF held-out	KB
AnnoForge	TBD	TBD	TBD
no attribution (one list)	TBD	TBD	TBD
whole-skill rewrites	TBD	TBD	TBD
`generate` only	TBD	TBD	TBD
no reviewer	TBD	TBD	TBD
+ test edits on held-in docs	TBD	TBD	TBD
+ consolidation	TBD	TBD	TBD
other document orders	TBD	TBD	TBD

Table 4: Planned matched-budget ablations: sequential gain on 0002–0005 and gain on the held-out set. All rows must be recomputed on this common protocol; the reported full-run gain over 0002–0010 is not a baseline for this subset. [G9]

Each time a sound core was lost, because rejected proposals are not revised. Without the reviewer (the TimeML and manifesto runs), the builder changes tools or rules in most rounds [G7: does admitting them help?].

6.5 Instruction growth without a clear increase in sequential gains

This trajectory shows no clear increase in gain with the number of preceding documents (Figure 2b): +10.1 on average for documents 2–6, +7.7 for 7–10 (Spearman $\rho = -0.62$, $p = 0.077$). The harness grows from 11.2 to 434 KB (200 in `generate`, 152 in `critique`, 81 in `document`), 13.6 KB per round. Of 1060 applied edits, 663 modify, 396 add and 1 delete (Appendix Table 7): nothing rewards a shorter harness. The correlation does not establish a flat learning curve, since document difficulty and order change along with the harness. Whether this pattern reflects what one document can teach, from interference among accumulated rules, or from length is open; held-out scores of H_1 to H_{10} and a consolidation pass under a size budget would separate these [G5, G6].

6.6 A weaker joint builder–annotator configuration

The gain holds for the procedural harness with the same models and for DeepSeek building and annotating without thinking (Table 3). With Qwen3.5-9B building and annotating over 13 documents, the learned harness is worse than the seed (−1.4 F on average; better on 3 documents), and the decline deepens (+0.6 in the first half, −3.0 in the second); two independent seed runs differ by 2.3 F per document, which limits interpretation of individual changes. The available reruns do not establish the significance of the trajectory. On agentic tasks, by contrast, a 9B model's harness updates were as useful as a frontier model's (Lin et al., 2026b). Whether builder or annotator matters more here is not yet separated [G8].

6.7 Ablations of the loop

Table 4 lists the ablations that test the design: whether attribution to a skill matters (one undivided problem list), whether typed edits matter (each round rewrites a skill in full, as guideline refinement does), whether the checking step is needed, whether the reviewer helps or only blocks, whether testing each edit before keeping it (Zhang et al., 2026a; Kim et al., 2026) closes the training-fit gap,

whether consolidation lifts the plateau, and how much document order matters [G9: to run].

6.8 Beyond UMR

The same loop builds a TimeML annotator for TempEval-3 (UzZaman et al., 2013) and a coder that assigns manifesto sentences one of the eight policy domains of the Manifesto Project scheme (Lehmann et al., 2026), with variant protocols that omit the model reviewer and permit rule and registry changes (Appendix F; finished baselines in Tables 12 and 13). On both test sets the seed harness is below the zero-shot prompt (TimeML event F1 74.5 vs. 80.7; manifesto accuracy 63.8 vs. 65.0), so the learned harness must first make up for its own seed. [G10, G11: learned harness on both test sets; sequential gains.]

7 Analysis

What the builder writes. In its first round the builder found 752 divergences in document 0001, read 250 of them (the cap), and sorted them into six patterns: form (colon-prefixed or capitalized values, abbreviated modal strengths, AMR role names such as `:time` for UMR's `:temporal`); named-entity types and tokens; discourse relations written as reified rolesets (`have-reason-91`) rather than edges; reported speech; Chinese granularity (which compounds split, which adverbs, demonstratives and classifiers keep a node); and document-level defaults for dates and temporal relations. These are the topics a human guideline spends its pages on, and they match the checks that improve on unseen documents (Appendix D shows edits verbatim).

What the builder asks for and cannot do. Each round may also list changes outside the instructions, which the loop does not apply. We coded all 130 (Appendix E). 78 concern the gold: conventions that differ between documents or annotators (the argument order of the concession roleset, whether title marks belong to a name, how names are split into tokens, which person type is used), usually with a decision not to learn from them yet, and outright errors such as undefined variables or an unbalanced bracket. 27 ask for deterministic code, such as normalizing attribute values or validating document-level relations against the graphs; one is the bracket fix we made by hand. 14 ask for more input to a skill (a running list of coreference chains, the genre) and 10 for evaluator changes. A builder with gold is thus also an auditor of gold, the moderation step of a human project; and by its own diagnosis much of what remains needs code and context, which the admission gate of §6.4 blocked every time.

How the harness goes wrong. Beyond the training-fit gap and unbounded growth, three failure modes recur. *Contradictions*: in round 2 the builder proposed a check forbidding aspect on discourse rolesets while its own round-1 rule and example required it; the reviewer caught this because it came as a tool, but edits to prose pass no such check. *Following the latest document*: after 0001 the instructions said a negated event is full-negative *without* `:polarity -`; the first round on 0002, whose gold marks polarity, made it require both. By R_{111}, disagreements between documents have become rules conditioned on genre ("in a biography only", "in a plot summary"). *Local regressions*: since no edit is tested before it is kept, a regression surfaces only on the next document. Testing edits on held-in documents (Zhang et al., 2026a; Wang et al., 2026) and sending gold conflicts to people may help with the last two [G9].

Cost. The seed harness costs about $0.08 per sentence at API list prices (three calls, cached prefixes). The learned harness reads up to 434 KB of instructions per sentence, mostly cached; building costs one call per sentence with an error and one planning call per round. [G12: cost and latency of H_1, H_4, H_{10}; cost of building; time to write the seed.]

8 Conclusion

In the reported Chinese UMR trajectory, supervised instruction revision improves a modular annotator by 9.0 sentence-graph F-score points over its seed on paired, previously untrained-on documents. Gains concentrate in convention-sensitive checks and are largely present at the generator stage. The same run exposes instruction growth, inconsistent rules and unsuccessful tool proposals. These findings describe one small, fixed-order study; they do not establish which design component causes the gains, a semantic-capability ceiling, or superiority to matched gold-budget alternatives. Held-out baselines, attribution and editing ablations, repeated trajectories, and builder–annotator swaps are the remaining tests of those claims.

Limitations

Scale and variance. The complete results are on one scheme and language, Chinese UMR, with 11 documents and 415 sentences in the sequential run; three documents have only 10–12 sentences. Each configuration was run once, in one document order; we report paired tests over documents and a sentence-level bootstrap, but not run-to-run variance of building or annotating [G-var].

No test before keeping an edit. ANNOFORGE keeps every edit that applies cleanly; its quality is measured only on the next document. Methods that test edits on held-in data may avoid some of the failure modes we report, and we have not compared them [G9].

Human influence. The seed harness, the judge's checks and the builder's prompts were written by the authors, and one rule step was changed by hand during the run (§4.1). The suggestions in §7 were coded by one person.

Models. The main run uses one closed model family for both annotating and building. Results with other models (Table 3) use a different harness and fewer documents, and DeepSeek ran without thinking.

Data. The leakage guard filters exact eight-character overlaps, not paraphrase, copying of shorter spans or document-specific knowledge. It consults held-out source sentences for rejection and can therefore provide an indirect constraint on training edits. API versions and runtime defaults may change, and exact historical backend, effort and cache settings must be audited before reproduction. The source texts of the documents may appear in pretraining data; [state whether the Chinese UMR gold is public]. Gold conventions differ between documents (§7), which bounds what any learner can reach.

Ethics Statement

Automating annotation can reduce the paid work available to annotators. We see the learned harness as a way to produce drafts and to audit gold, with people deciding conventions; its suggestions about inconsistent gold are addressed to them. Model calls use commercial APIs or open-weight deployments. [Verify the exact data release, license, permitted API use and redistribution terms; identify what source text and gold were sent to each service.] We do not infer those permissions from the presence of a local data checkout.

Generative AI assisted author-side simulated review and manuscript editing. It did not supply new experimental results; this assistance is not formal ARR peer review.

References

Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziems, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

Anonymous. 2026. Decision orchestration: Learning annotation procedures from a handful of gold documents. Under review.

Adrien Bibal, Nathaniel Gerlek, Goran Muric, Elizabeth Boschee, Steven C. Fincke, Mike Ross, and Steven N. Minton. 2025. Automating annotation guideline improvements using LLMs: A case study. In *Proceedings of Context and Meaning: Navigating Disagreements in NLP Annotation*, pages 129–144, Abu Dhabi, UAE. International Committee on Computational Linguistics.

Julia Bonn, Matthew Buchholz, Jayeol Chun, Andrew Cowell, William Croft, Lukas Denk, Sijia Ge, Jan Hajič, Kenneth Lai, James H. Martin, Skatje Myers, Alexis Palmer, Martha Palmer, Claire Benét Post, James Pustejovsky, Kristine Stenzel, Haibo Sun, Zdeňka Urešová, Rosa Vallejos, and 4 others. 2024. Building a broad infrastructure for uniform meaning representations. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2537–2547, Torino, Italia. ELRA and ICCL.

Eric Brill. 1995. Transformation-based error-driven learning and natural language processing: A case study in part-of-speech tagging. *Computational Linguistics*, 21(4):543–565.

Tianle Cai, Xuezhi Wang, Tengyu Ma, Xinyun Chen, and Denny Zhou. 2024. Large language models as tool makers. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Jayeol Chun and Nianwen Xue. 2024. Uniform meaning representation parsing as a pipelined approach. In *Proceedings of TextGraphs-17: Graph-based Methods for Natural Language Processing*, pages 40–52.

A. P. Dawid. 1984. Present position and potential developments: Some personal views: Statistical theory:

The prequential approach. *Journal of the Royal Statistical Society. Series A (General)*, 147(2):278.

Allyson Ettinger, Jena Hwang, Valentina Pyatkin, Chandra Bhagavatula, and Yejin Choi. 2023. "you are an expert linguistic annotator": Limits of LLMs as analyzers of abstract meaning representation. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 8250–8263.

Marcio Fonseca and Shay B. Cohen. 2024. Can large language models follow concept annotation guidelines? a case study on scientific and financial domains. In *Findings of the Association for Computational Linguistics: ACL 2024*, pages 8027–8042.

João Gama, Raquel Sebastião, and Pedro Pereira Rodrigues. 2013. On evaluating stream learning algorithms. *Machine Learning*, 90(3):317–346.

Fabrizio Gilardi, Meysam Alizadeh, and Maël Kubli. 2023. ChatGPT outperforms crowd workers for text-annotation tasks. *Proceedings of the National Academy of Sciences*, 120(30):e2305016120.

Andrew Halterman and Katherine A. Keith. 2026. Codebook LLMs: Evaluating LLMs as measurement tools for political science concepts. *Political Analysis*, 34(2):188–204.

Xingwei He, Zhenghao Lin, Yeyun Gong, A-Long Jin, Hang Zhang, Chen Lin, Jian Jiao, Siu Ming Yiu, Nan Duan, and Weizhu Chen. 2024. AnnoLLM: Making large language models to be better crowdsourced annotators. In *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 6: Industry Track)*, pages 165–190.

Zeyu He, Zhuqian Zhou, Kirk Vanacore, Rene F. Kizilcec, and Ting-Hao 'Kenneth' Huang. 2026. Experts rise where LLMs disagree: Using cross-model disagreement to target expert effort in LLM codebook revision for large-scale annotation. arXiv:2609.26926. *Preprint*, arXiv:2609.26926.

Shengran Hu, Cong Lu, and Jeff Clune. 2025. Automated design of agentic systems. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large language models cannot self-correct reasoning yet. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Shizhou Huang, Bo Xu, Yang Yu, Changqun Li, and Xin Alex Lin. 2025. GuideNER: Annotation guidelines are better than examples for in-context named entity recognition. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 39, pages 24159–24166.

Ryo Kamoi, Yusen Zhang, Nan Zhang, Jiawei Han, and Rui Zhang. 2024. When can LLMs actually correct their own mistakes? a critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440.

Omar Khattab, Arnav Singhvi, Paridhi Maheshwari, Zhiyuan Zhang, Keshav Santhanam, Sri Vardhamanan, Saiful Haq, Ashutosh Sharma, Thomas T. Joshi, Hanna Moazam, Heather Miller, Matei Zaharia, and Christopher Potts. 2024. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Kon Woo Kim, Rezarta Islamaj, Jin-Dong Kim, Florian Boudin, and Akiko Aizawa. 2025. Repurposing annotation guidelines to instruct LLM annotators: A case study. In *International Conference on Applications of Natural Language to Information Systems (NLDB 2025), Springer LNCS*, pages 140–151.

Kon Woo Kim, Jin-Dong Kim, and Akiko Aizawa. 2026. Refining and reusing annotation guidelines for LLM annotation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 37951–37964, San Diego, California, United States. Association for Computational Linguistics.

Yoonho Lee, Roshen Nair, Qizheng Zhang, Kangwook Lee, Omar Khattab, and Chelsea Finn. 2026. Meta-harness: End-to-end optimization of model harnesses. arXiv:2603.28052. *Preprint*, arXiv:2603.28052.

Pola Lehmann, Simon Franzmann, Denise Al-Gaddooa, Christoph Ivanusch, Jirka Lewandowski, Felicia Riethmüller, Svenja Samstag, and Lisa Zehnter. 2026. Manifesto corpus. version 2026-1.

Jiahang Lin, Shichun Liu, Chengjun Pan, Lizhi Lin, Shihan Dou, Zhiheng Xi, Xuanjing Huang, Hang Yan, Zhenhua Han, Tao Gui, and Yu-Gang Jiang. 2026a. Agentic harness engineering: Observability-driven automatic evolution of coding-agent harnesses. arXiv:2604.25850. *Preprint*, arXiv:2604.25850.

Minhua Lin, Juncheng Wu, Zijun Wang, Zhan Shi, Yisi Sang, Bing He, Zewen Liu, Tianxin Wei, Zongyu Wu, Zhiwei Zhang, Dakuo Wang, Xiang Zhang, Benoit Dumoulin, Cihang Xie, Yuyin Zhou, Suhang Wang, and Hanqing Lu. 2026b. Harness updating is not harness benefit: Disentangling evolution capabilities in self-evolving LLM agents. arXiv:2605.30621. *Preprint*, arXiv:2605.30621.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Emma Markle, Javier Gutierrez Bach, and Shira Wein. 2026. SETUP: Sentence-level english-to-uniform meaning representation parser. In *Proceedings of LREC 2026*.

Krista Opsahl-Ong, Michael J Ryan, Josh Purtell, David Broman, Christopher Potts, Matei Zaharia, and Omar Khattab. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366.

Martha Palmer, Daniel Gildea, and Paul Kingsbury. 2005. The proposition bank: An annotated corpus of semantic roles. *Computational Linguistics*, 31(1):71–106.

Chaoxu Pang, Yixuan Cao, Qiang Ding, and Ping Luo. 2023. Guideline learning for in-context information extraction. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 15372–15389, Singapore. Association for Computational Linguistics.

Reid Pryzant, Dan Iter, Jerry Li, Yin Lee, Chenguang Zhu, and Michael Zeng. 2023. Automatic prompt optimization with "gradient descent" and beam search. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 7957–7968.

James Pustejovsky, José Castaño, Robert Ingria, Roser Saurí, Robert Gaizauskas, Andrea Setzer, Graham Katz, and Dragomir Radev. 2003. TimeML: Robust specification of event and temporal expressions in text. In *AAAI Spring Symposium 2003 (AAAI Technical Report SS-03-07)*.

Cheng Qian, Chi Han, Yi Fung, Yujia Qin, Zhiyuan Liu, and Heng Ji. 2023. CREATOR: Tool creation for disentangling abstract and concrete reasoning of large language models. In *Findings of the Association for Computational Linguistics: EMNLP 2023*, pages 6922–6939.

Oscar Sainz, Iker García-Ferrero, Rodrigo Agerri, Oier Lopez de Lacalle, German Rigau, and Eneko Agirre. 2024. GoLLIE: Annotation guidelines improve zero-shot information-extraction. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*.

Prateek Singh. 2026. MAGE: Understanding stability-performance trade-offs in multi-component prompt optimization. arXiv:2607.11944. *Preprint*, arXiv:2607.11944.

Saurabh Srivastava, Sweta Pati, and Ziyu Yao. 2025. Instruction-tuning LLMs for event extraction with annotation guidelines. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 13055–13071.

Haibo Sun, Jayeol Chun, and Nianwen Xue. 2025. AnCast++: Document-level evaluation of graph-based meaning representations. In *Findings of the Association for Computational Linguistics: ACL 2025*, pages 19642–19654.

Haibo Sun and Nianwen Xue. 2024. Anchor and broadcast: An efficient concept alignment approach for evaluation of semantic graphs. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 1052–1062, Torino, Italia. ELRA and ICCL.

Haibo Sun, Nianwen Xue, Jin Zhao, Liulu Yue, Yao Sun, Keer Xu, and Jiawei Wu. 2024. Chinese UMR annotation: Can LLMs help? In *Proceedings of the Fifth International Workshop on Designing Meaning Representations @ LREC-COLING 2024*, pages 131–139, Torino, Italia. ELRA and ICCL.

Mirac Suzgun, Mert Yuksekgonul, Federico Bianchi, Dan Jurafsky, and James Zou. 2026. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106.

Zhen Tan, Dawei Li, Song Wang, Alimohammad Beigi, Bohan Jiang, Amrita Bhattacharjee, Mansooreh Karami, Jundong Li, Lu Cheng, and Huan Liu. 2024. Large language models for data annotation and synthesis: A survey. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 930–957.

Naushad UzZaman, Hector Llorens, Leon Derczynski, James Allen, Marc Verhagen, and James Pustejovsky. 2013. SemEval-2013 task 1: TempEval-3: Evaluating time expressions, events, and temporal relations. In *Second Joint Conference on Lexical and Computational Semantics (*SEM), Volume 2: Proceedings of the Seventh International Workshop on Semantic Evaluation (SemEval 2013)*, pages 1–9, Atlanta, Georgia, USA. Association for Computational Linguistics.

Jens E. L. Van Gysel, Meagan Vigus, Jayeol Chun, Kenneth Lai, Sarah Moeller, Jiarui Yao, Tim O'Gorman, Andrew Cowell, William Croft, Chu-Ren Huang, Jan Hajič, James H. Martin, Stephan Oepen, Martha Palmer, James Pustejovsky, Rosa Vallejos, and Nianwen Xue. 2021. Designing a uniform meaning representation for natural language processing. *KI - Künstliche Intelligenz*, 35(3–4):343–360.

Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. 2024. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*.

Wenxiao Wang, Priyatham Kattakinda, and Soheil Feizi. 2026. Do agent optimizers compound? a continual-learning evaluation on terminal-bench 2.0. arXiv:2607.14004. *Preprint*, arXiv:2607.14004.

Georg Wölflein, Dyke Ferber, Daniel Truhn, Ognjen Arandjelović, and Jakob Nikolas Kather. 2025. LLM agents making agent tools. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 26092–26130.

Peng Xia, Rujun Han, Zifeng Wang, Yanfei Chen, Yufan Zhuang, Yoonho Lee, Chengsong Huang, Han Yu, Zhongying CuiZhu, Yifei Ming, Huaxiu Yao, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. 2026. RRSI: Regularized recursive self-improvement of agent harnesses. arXiv:2609.24972. *Preprint*, arXiv:2609.24972.

Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *The Twelfth International Conference on Learning Representations (ICLR)*.

Boqin Yuan, Xiaoyi Gu, Fiona Li, Chang Wan, Angel Hsing-Chi Hwang, and Jieyu Zhao. 2026. CALICO: A human-centered, codebook-aligned system for annotation. arXiv:2609.14726. *Preprint*, arXiv:2609.14726.

Mert Yuksekgonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639(8055):609–616.

Hangfan Zhang, Shao Zhang, Kangcong Li, Chen Zhang, Yang Chen, Yiqun Zhang, Lei Bai, and Shuyue Hu. 2026a. Self-harness: Harnesses that improve themselves. arXiv:2606.09498. *Preprint*, arXiv:2606.09498.

Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. 2026b. Darwin Gödel machine: Open-ended evolution of self-improving agents. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, Bingnan Zheng, Bang Liu, Yuyu Luo, and Chenglin Wu. 2025. AFlow: Automating agentic workflow generation. In *The Thirteenth International Conference on Learning Representations (ICLR)*.

Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, Urmish Thakker, James Zou, and Kunle Olukotun. 2026c. Agentic context engineering: Evolving contexts for self-improving language models. In *The Fourteenth International Conference on Learning Representations (ICLR)*.

Boyuan Zheng, Michael Y. Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, and Yu Su. 2025. SkillWeaver: Web agents can self-improve by discovering and honing skills. arXiv:2504.07079. *Preprint*, arXiv:2504.07079.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *The Eleventh International Conference on Learning Representations (ICLR)*.

Caleb Ziems, William Held, Omar Shaikh, Jiaao Chen, Zhehao Zhang, and Diyi Yang. 2024. Can large language models transform computational social science? *Computational Linguistics*, 50(1):237–291.

A The Judge

AnCast++ aligns each predicted node to at most one gold node. The judge then scores 25 checks in 11 groups over the aligned graphs; each check yields units with a verdict (correct, partial, wrong, missing, extra) and a credit, and reports F over its units. Three checks never fire on this corpus (granularity, the modal-strength attribute, which is moved to the document level, and Wikipedia links).

Group	Checks
a nodes	`event_found`, `event_concept`, `entity_found`, `entity_concept`, `granularity`
b senses	`event_sense`
c attributes	`aspect`, `modstr`, `polarity`, `refer`, `other_attributes`
d arguments	`core_args_event`, `core_args_abstract`, `peripheral_event`, `peripheral_entity`
e role labels	`core_labels`, `peripheral_labels`
f names	`name_tokens`, `wiki`
g reentrancy	`reentrancy`
h–k document	`temporal`, `modal`, `coref_chain`, `coref_subset`, `doc_format`

Table 5: The judge's checks.

B Records of the Sequential Run

Table 6 lists every round, Table 7 the applied edits by skill and action, Table 8 the per-check scores behind Figure 3, Table 9 what the checking step adds, Table 10 the variant with five checking steps, and Table 11 the matched comparison with the procedural harness.

C Admission of Tools and Steps

Machine checks. A new step must load in the registry, attach after an existing step of the right chain, and keep its instruction prefix constant across calls. A new tool may import only `re`, `json`, `math`, `itertools`, `collections`, `functools`, `unicodedata`, `string` and `penman`;

Round	Probl.	D-lvl	Gen.	Crit.	Doc.	Rej.	Proposal
R_{11}	752	187	3	8	6	3	check_umr_graph (rej.)
R_{12}	610	183	18	15	5	4	check_umr_form (rej.)
R_{13}	516	175	31	18	3	0	–
R_{21}	775	254	26	18	4	0	–
R_{22}	693	262	34	24	6	1	–
R_{23}	623	266	14	14	7	0	–
R_{31}	407	114	16	9	4	0	–
R_{32}	314	105	34	27	2	0	–
R_{33}	250	111	17	14	5	1	check_doc_relations (rej.)
R_{41}	780	237	13	11	2	0	–
R_{42}	698	245	24	13	3	2	–
R_{43}	625	231	17	11	3	1	–
R_{51}	463	144	16	12	4	0	–
R_{52}	404	151	25	24	7	1	–
R_{53}	343	117	36	27	4	1	–
R_{61}	440	123	24	18	2	0	–
R_{62}	379	149	20	12	4	1	–
R_{63}	311	122	12	12	3	0	–
R_{71}	692	273	11	7	2	0	–
R_{72}	548	251	11	13	7	0	–
R_{73}	524	256	15	18	5	1	–
R_{81}	132	40	14	7	4	1	–
R_{82}	96	33	17	7	5	0	–
R_{83}	73	28	3	8	5	0	–
R_{91}	109	39	15	9	4	0	–
R_{92}	67	33	12	12	9	1	–
R_{93}	43	29	5	7	7	1	–
R_{101}	188	62	2	7	2	0	–
R_{102}	92	42	12	8	10	0	–
R_{103}	67	36	10	13	9	0	–
R_{111}	258	80	2	3	2	0	–

Table 6: Every improvement round of the sequential run: divergences found (of them document-level), edits applied per skill, edits rejected, and tool or step proposals.

Skill	Modify	Add	Delete	Total
`generate`	333	176	0	509
`critique`	255	151	0	406
`document`	75	69	1	145
All	663	396	1	1060

Table 7: Applied edits over 31 improvement rounds (19 further edits were rejected: anchor not found, copied document text, or depending on a rejected tool).

an AST check rejects `open`, `eval`, `exec`, `getattr` and access to underscored attributes, and requires exactly one public function with type annotations and a docstring. The proposal brings at least three tests, run in an isolated subprocess, all of which must pass. Nothing in a proposal may share eight characters with a corpus sentence.

Reviewer. A separate call of the builder model receives the pipeline, the proposal, the machine-check results, the problems it addresses, all problem counts, the judge scores and the current skills. It approves only if all six criteria clearly hold and rejects when in doubt: (1) the problems are many, costly and will recur; (2) an edit to the skill that owns the aspect could not do as well; (3) one narrow responsibility that no skill has, in the right

Check	Seed	Learned	Δ	Docs ↑
`c_refer`	0.53	0.78	+0.25	5/6
`i_modal`	0.40	0.59	+0.19	9/9
`e_peripheral_labels`	0.67	0.86	+0.18	9/9
`d_core_args_abstract`	0.30	0.43	+0.13	7/9
`d_peripheral_entity`	0.43	0.56	+0.13	8/9
`d_peripheral_event`	0.35	0.47	+0.12	8/9
`a_entity_concept`	0.71	0.82	+0.11	8/9
`h_temporal`	0.29	0.39	+0.09	6/9
`k_doc_format`	0.91	0.98	+0.08	9/9
`g_reentrancy`	0.41	0.48	+0.07	8/9
`j_coref_chain`	0.26	0.32	+0.05	6/9
`f_name_tokens`	0.55	0.59	+0.04	5/9
`a_event_concept`	0.81	0.85	+0.04	6/9
`a_entity_found`	0.92	0.95	+0.03	9/9
`e_core_labels`	0.84	0.87	+0.02	5/9
`a_event_found`	0.95	0.97	+0.02	6/9
`c_other_attributes`	0.70	0.71	+0.02	5/9
`c_aspect`	0.68	0.68	+0.00	6/9
`d_core_args_event`	0.65	0.65	+0.00	5/9
`b_event_sense`	0.90	0.90	+0.00	4/9
`c_polarity`	0.68	0.57	−0.11	3/8

Table 8: Judge score per check on unseen documents, seed vs. learned harness (mean over the paired documents; "Docs ↑" counts documents where the learned harness is higher).

Annotated by	Mean	Docs > 0	Range
Seed H_0	+1.0	8/10	−0.1 to +2.3
Learned, unseen doc.	+0.5	8/10	−0.4 to +1.4
Trained on the doc.	+1.7	10/10	+0.3 to +4.4

Table 9: What the `critique` step adds to the `generate` draft (AnCast++ sentence F, sentence graphs only, before the document layer).

place, not fighting another skill; (4) worth its cost: a step is one more call per sentence and a new chance to break a correct graph, a tool must do what a model cannot do reliably (an exact lookup, a computation, a well-formedness check); (5) correct and safe, consistent with the other skills, with real tests; (6) no leakage.

D Edits, Verbatim

Two edits from the first round on document 0001, as applied (line breaks and some list items omitted). Each was written in general terms; the guard rejects any text sharing eight characters with a corpus sentence.

Added to `critique`, group e (role labels).

```
- AMR labels: ‘:time‘ must be
‘:temporal‘ and ‘:poss‘ must be
‘:possessor‘; - have-org-role-91
numbered the AMR way. In UMR, :ARG1 is
the holder, :ARG2 the organization or
country and :ARG3 the title;
```

	Doc	Version	generate	+5 checks	Δ
trained	0001	R_{13}	65.8	75.0	+9.3
	0002	R_{23}	67.0	75.9	+8.9
unseen	0002	R_{13}	57.3	59.7	+2.4
	0004	R_{13}	49.1	50.0	+0.9
	0005	R_{13}	58.9	57.6	−1.3
	0004	R_{23}	50.4	51.9	+1.5
	0005	R_{23}	58.6	57.5	−1.1

Table 10: A variant with five checking steps after `generate` (concepts, arguments, attributes, entities, reentrancy), improved by the same loop. Sentence F after `generate` and after the five checks, on the document the version was trained on and on documents it had not seen.

Held-out doc	Procedural	Ours	Few-shot
0005	55.7	58.5	TBD
0006	49.5	TBD	TBD
0007	62.0	TBD	TBD

Table 11: Trained on documents 0001–0004 with the same models, tested on 0005–0007, sentence F. The procedural harness (R_{42}) has had 10 rounds (two on each of 0003 and 0004), ours (H_4) 12.

> `have-rel-role-91 is numbered the same way. identity-91 has :ARG1 and :ARG2 and no :ARG0. have-org-role-92 and have-role-91 are wrong;`
> *Reason given:* P8 (`:poss` vs. `:possessor`), P15, P188, P192, P194, P203–P205, P208, P212–P220, P173, P177, P239.

Replacing a rule of `document`.

> `- an absolute date in the past (a date-entity with a year: *in 1979*) is 'document-creation-time' 'before' it; a relative or vague time expression (*today*, *last week*, *next year*) is 'document-creation-time' 'depends-on' it; - an event with an explicit time expression is 'overlap' that expression's node (the date is the source, the event the target); - between two events, use 'before' or 'after' only when the text marks an order (then, later, as a result). [. . .]`
> *Reason given:* P245 (absolute past date → before), P246 (date overlap event), P248 (overlap by default), P250 (no temporal parent for role predicates).

A reversal. R_{13}, `generate`: "A negated event (未经 + a procedure, 从不 V, 没有V) is full-negative, with no :polarity -." R_{21}, after the first round on 0002: "A negated event (未经 + a procedure, 从不 V, 没有V) is full-negative, together with :polarity -."

	TIMEX3			EVENT		TLINK
System	strict	relaxed	value	F1	class	F1
HeidelTime (2013)	81.3	90.3	77.6	–	–	–
ClearTK (2013)	82.7	90.2	64.7	77.3	65.4	29.8
TIPSem (2013)	81.6	84.9	65.3	82.9	75.6	42.4
Zero-shot prompt	85.4	92.9	84.6	80.7	65.5	39.8
Few-shot (10 docs)	TBD	TBD	TBD	TBD	TBD	TBD
Seed H_0	81.1	90.2	79.0	74.5	61.9	37.9
Learned harness	TBD	TBD	TBD	TBD	TBD	TBD

Table 12: TempEval-3 platinum test (20 documents), official scorer. 2013 systems rescored from their released outputs (UzZaman et al., 2013); "–": the system does not produce this layer.

E The Builder's Suggestions

Each round's plan may list changes the loop cannot apply. We coded all 130 (file `data/umr/suggestions_coded.csv`) into five codes: *gold* (78): a convention that differs between documents or annotators, or an error in gold, usually with a decision not to learn from it; *rule code* (27): a deterministic change to a rule step, the output handler or a validator; *context* (14): more input to a skill, such as earlier sentences, a running list of coreference chains, or the document's genre; *evaluator* (10): a change to scoring, for instance normalizing equivalent quantity forms; *other* (1). Examples, verbatim: "Check the gold argument order of but-91 for concessives [. . .] The prompts follow the majority" (gold, round 1); "Make final_graph (or the output handler) normalise attribute values before choosing a graph" (rule code, round 1); "Give the document step a running list of coreference chains for the whole document so far" (context, document 0006); "Have the scorer report AnCast node-matching cascades separately from real concept errors" (evaluator, document 0003). [G13: a second coder and agreement.]

F Other Schemes and Models

TimeML. Test: the 20 TempEval-3 platinum documents with the official scorer (UzZaman et al., 2013); training documents in a fixed order from TimeBank and AQUAINT (TBAQ-cleaned). The harness marks time expressions and events per sentence (`mark`, with a normalization tool for values, then `critique`), then writes TLINKs per document (`link`, `link_critique`, with a link-checking tool). The 2013 systems are rescored from their released outputs with the same scorer, which reproduces their published numbers. Zero-shot: the seed's

System	Acc.	Macro-F1
Majority class	33.4	6.3
TF-IDF + LR, same 1,200 sentences	42.9	22.9
TF-IDF + LR, 33k sentences	62.5	50.3
ManifestoBERTa*	72.1	59.2
Zero-shot prompt	65.0	56.0
Few-shot, 30 retrieved examples	67.8	58.5
Seed harness H_0	63.8	54.5
Learned harness	TBD	TBD

Table 13: Manifesto coding into eight policy domains, 800 test sentences. *Trained on the Manifesto Corpus, which these sentences come from: a reference, not a fair comparison.

instructions and the whole document in one call; few-shot: the same with the ten training documents and their gold as examples.

Manifesto coding. English sentences of electoral programs with one of eight domains of the Manifesto Project coding scheme (dataset `siacus/manifesto` on HuggingFace). The dataset has no document identifiers, so a "document" is a random batch of 120 sentences: ten training batches and one test set of 800 sentences. The harness is `classify` with a codebook lookup tool, then `critique`. Few-shot uses the 30 training sentences most similar to the input (TF-IDF).

Admission in these runs. Both use machine checks only, without the reviewer, and also allow the builder to rewrite rule code, add or remove steps, and change the registry; a change that fails its checks is dropped.

Planned matched TimeML evaluation. A new evaluation will use one frozen Haiku version for all annotation stages and for the zero-shot, few-shot, seed and adapted conditions, with Opus for supervised construction and review. The main adaptation protocol will match UMR's restricted edits and admission checks; the historical wider-edit protocol will be a separate condition. Existing TimeML scores do not evaluate this configuration. Gold exposure, annotation repetitions, construction cost and failures will be reported separately [G10]. Manifesto remains an optional task-boundary experiment.

Qwen3.5-9B. Served with vLLM, thinking on, as both annotator and builder, on the procedural harness, sequentially over documents 0008–0035. Its anchors are often inexact (with strict matching, 1 of its first 15 edits applied), so its runs use fuzzy anchors (a unique line with similarity at least 0.85). Two independent runs of the seed estimate annotation noise.

DeepSeek. The same model annotates and builds, on the procedural harness: two rounds each on documents 0001 and 0004, test on 0002, 0003 and 0005–0007; thinking was off in these runs.

G Reproduction and Exposure Audit

The bundled score snapshots and generated tables support the reported trajectory, but are not a complete reproduction package. Before submission, record exact code and scorer commits, gold-file checksums and preprocessing changes, model identifiers and run dates, historical backend/thinking/effort, concurrency, cache policy, and failures/exclusions [G14]. Separate each run's manual seed-design exposure, builder training, regression/development selection, overlap-filter inputs and test scoring. The 20-document planned split is specific to the main UMR harness; other experiments used some of these documents. Match training and inference information, annotation rounds and costs when completing G1–G4. The current procedural comparison has unequal round counts and is exploratory. Anonymize artifact locations for review while retaining technical version identifiers.